16. WRITE THE PYTHON PROGRAM TO IMPLEMENT BFS.

Aim

To write a Python program to implement the Breadth-First Search (BFS) algorithm to traverse a graph.

Algorithm

1. Start the program.
2. Create a graph using a dictionary.
3. Select the starting node.
4. Create an empty queue and add the starting node.
5. Mark the starting node as visited.
6. Remove the first node from the queue and display it.
7. Add all its unvisited neighbouring nodes to the queue.
8. Repeat steps 6 and 7 until the queue is empty.
9. Stop the program.

Python Code

```
# BFS (Breadth-First Search)

from collections import deque

# Create a graph
graph = {
  'A': ['B', 'C'],
  'B': ['D', 'E'],
  'C': ['F'],
  'D': [],
  'E': [],
  'F': []
}

def bfs(graph, start):
```

```
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

# Start BFS from node A
print("BFS Traversal:")
bfs(graph, 'A')
```

Output:

```
                    visited.add(neigh
                    queue.append(neig

# Start BFS from node A
print("BFS Traversal:")
bfs(graph, 'A')

BFS Traversal:
A B C D E F
```

17. WRITE THE PYTHON PROGRAM TO IMPLEMENT DFS.

Aim

To write a Python program to implement Depth-First Search (DFS) to traverse a graph.

Algorithm

1. Start the program.
2. Create a graph using a dictionary.

3. Select the starting node A.
4. Mark the node as visited and display it.
5. Visit an unvisited neighbouring node.
6. Repeat until all reachable nodes are visited.
7. Stop the program.

Python Code

```
# DFS (Depth-First Search)

graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}

def dfs(node, visited):
   if node not in visited:
      visited.add(node)
      print(node, end=" ")

      for neighbour in graph[node]:
         dfs(neighbour, visited)

print("DFS Traversal:")
dfs('A', set())
```

Output:

```
print("DFS Traversal:")
dfs('A', set())
```

```
DFS Traversal:
A B D E C F
```

18. PYTHON PROGRAM TO IMPLEMENT TRAVELLING SALESMAN PROBLEM (TSP)

Aim

To write a Python program to find the minimum-distance tour for the Travelling Salesman Problem.

Algorithm

1. Start the program.
2. Define the distance between cities.
3. Select the starting city A.
4. Generate all possible orders of the remaining cities.
5. Calculate the total distance of each tour, including the return to A.
6. Select the tour with the minimum distance.
7. Display the best tour and its distance.
8. Stop the program.

Python Code

```
# Travelling Salesman Problem

from itertools import permutations

cities = ['A', 'B', 'C', 'D']

distance = [
  [0, 10, 15, 20],
  [10, 0, 35, 25],
  [15, 35, 0, 30],
  [20, 25, 30, 0]
```

```
]

start = 0
other_cities = [1, 2, 3]

min_distance = float('inf')
best_path = []

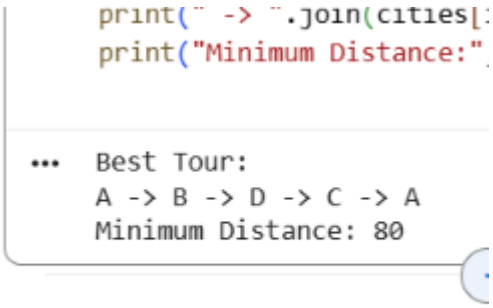
for path in permutations(other_cities):
    tour = (start,) + path + (start,)
    total = 0

    for i in range(len(tour) - 1):
        total += distance[tour[i]][tour[i + 1]]

    if total < min_distance:
        min_distance = total
        best_path = tour

print("Best Tour:")
print(" -> ".join(cities[i] for i in best_path))
print("Minimum Distance:", min_distance)
```

Output:



19. PYTHON PROGRAM TO IMPLEMENT A* ALGORITHM

Aim

To write a Python program to find the shortest path between a starting node and a goal node using the A* search algorithm.

Algorithm

1. Start the program.
2. Define the graph with distances between nodes.
3. Define the heuristic values for each node.
4. Select the starting node and goal node.
5. Calculate $f(n)=g(n)+h(n)$, where $g(n)$ is the cost so far and $h(n)$ is the estimated cost to the goal.
6. Select the node with the lowest $f(n)$ value.
7. Repeat until the goal node is reached.
8. Display the shortest path and its cost.
9. Stop the program.

Python Code

```
# A* Search Algorithm

import heapq

graph = {
   'A': {'B': 1, 'C': 3},
   'B': {'D': 3, 'E': 5},
   'C': {'D': 1},
   'D': {'E': 2, 'G': 4},
   'E': {'G': 1},
   'G': {}
}

# Estimated distance to goal G
h = {
```

```
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 3,
    'E': 1,
    'G': 0
}

def astar(start, goal):
    queue = [(h[start], 0, start, [start])]
    visited = set()

    while queue:
        f, cost, node, path = heapq.heappop(queue)

        if node == goal:
            return path, cost

        if node in visited:
            continue

        visited.add(node)

        for neighbour, distance in graph[node].items():
            if neighbour not in visited:
                new_cost = cost + distance
                new_f = new_cost + h[neighbour]
                heapq.heappush(
                    queue,
                    (new_f, new_cost, neighbour,
                     path + [neighbour])
```

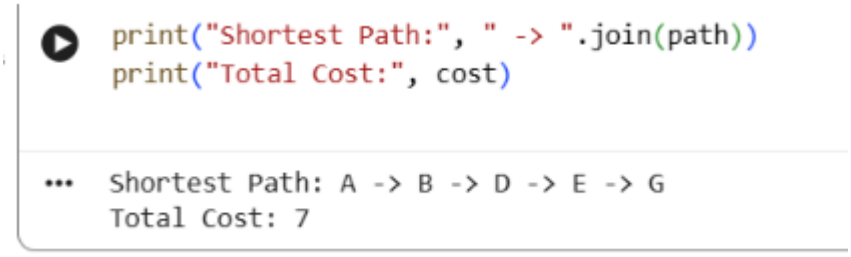
```
            )

    return None, float('inf')

path, cost = astar('A', 'G')

print("Shortest Path:", " -> ".join(path))
print("Total Cost:", cost)
```

Output:



20. PYTHON PROGRAM FOR MAP COLORING USING CSP

Aim

To write a Python program to solve the Map Coloring problem using a Constraint Satisfaction Problem (CSP) approach.

Algorithm

1. Start the program.
2. Define the regions and their neighbouring regions.
3. Define the available colors: Red, Green, and Blue.
4. Assign a color to each region.
5. Check that neighbouring regions do not have the same color.
6. If a conflict occurs, try another color.
7. Continue until all regions are colored.
8. Display the color assigned to each region.
9. Stop the program.

Python Code

```
# Map Coloring using CSP
```

```
regions = ['A', 'B', 'C', 'D']

colors = ['Red', 'Green', 'Blue']

# Neighbouring regions
neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

assignment = {}

def is_safe(region, color):
    for neighbour in neighbors[region]:
        if assignment.get(neighbour) == color:
            return False
    return True

def solve(index):
    if index == len(regions):
        return True

    region = regions[index]

    for color in colors:
        if is_safe(region, color):
            assignment[region] = color
```

```
        if solve(index + 1):
            return True

        del assignment[region]

    return False

if solve(0):
    print("Map Coloring Solution:")
    for region in regions:
        print(region, ":", assignment[region])
else:
    print("No solution found")
```

Output:

```
        return False

    if solve(0):
        print("Map Coloring Solution:")
        for region in regions:
            print(region, ":", assignment[region])
    else:
        print("No solution found")
```

```
Map Coloring Solution:
A : Red
B : Green
C : Blue
D : Red
```